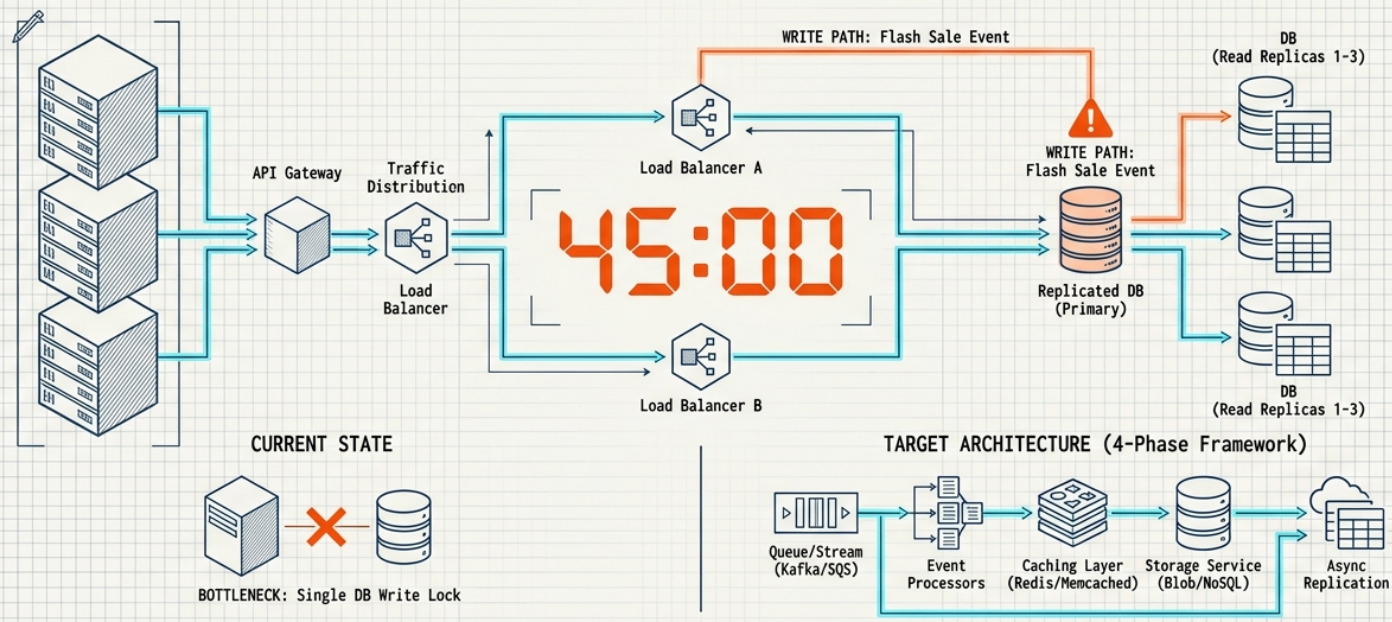
The System Design Playbook: Mastering the 45-Minute Interview

A repeatable 4-phase framework, stress-tested against the brutal write-heavy architecture of a Flash Sale.



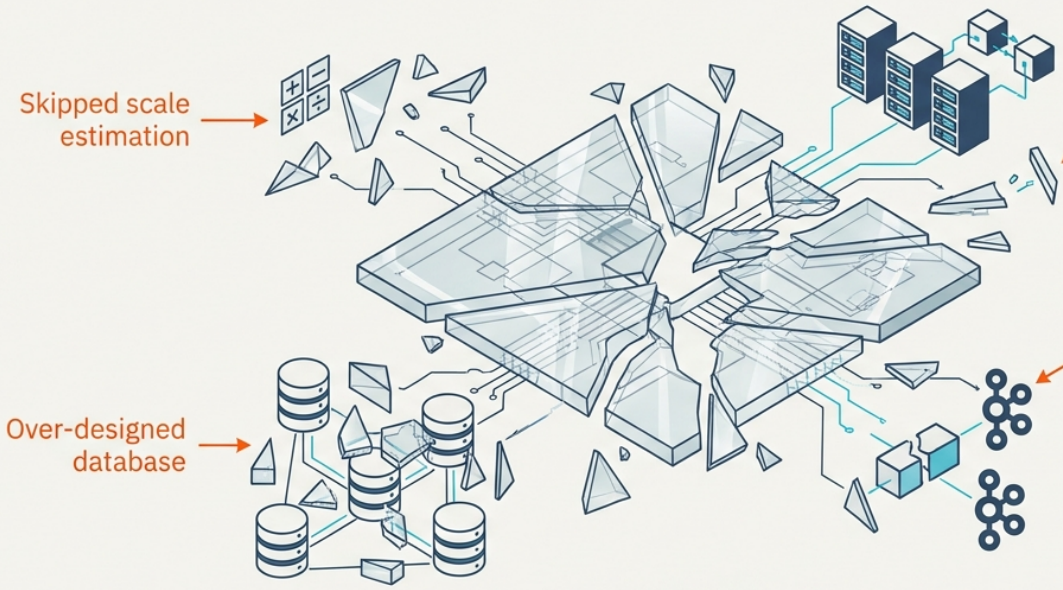
CURRENT STATE

BOTTLENECK: Single DB Write Lock

TARGET ARCHITECTURE (4-Phase Framework)

PHASE 1: Requirements Gathering (5 min) > PHASE 2: High-Level Design (10 min) > PHASE 3: Deep Dive (20 min) > PHASE 4: Wrap-up & Trade-offs (5 min)

“The interviewer evaluates your ability to structure ambiguity, not your memory”



Without a structured process, candidates spend 20 minutes on load balancers, forget to calculate QPS, and run out of time before discussing trade-offs.

The goal is a repeatable mental process, whether designing Twitter, Uber, or a ticket booking system.

The 45-Minute Framework: A Chronological Process

Master the chronological process to survive and thrive. The Flash Sale—a contintion-heavy, write-heavy problem that breaks standard e-commerce architectures—will act as our ultimate stress test.

Phase 1 Playbook: Focus the Scope

Functional Requirements

What does the system actually do?

- Limit initial scope to 3–4 core user journeys
- Core features and interactions
- User inputs and system outputs

Non-Functional Requirements

How well must it perform?

- Latency and Availability
- Consistency and Durability
- Geographic Distribution

GOLDEN RULE: Ask only questions that materially alter the architecture. (Who are the users? How many? What is the read/write ratio?)

Execution: Scoping the 10-Million User Flash Sale

The Features (Functional)

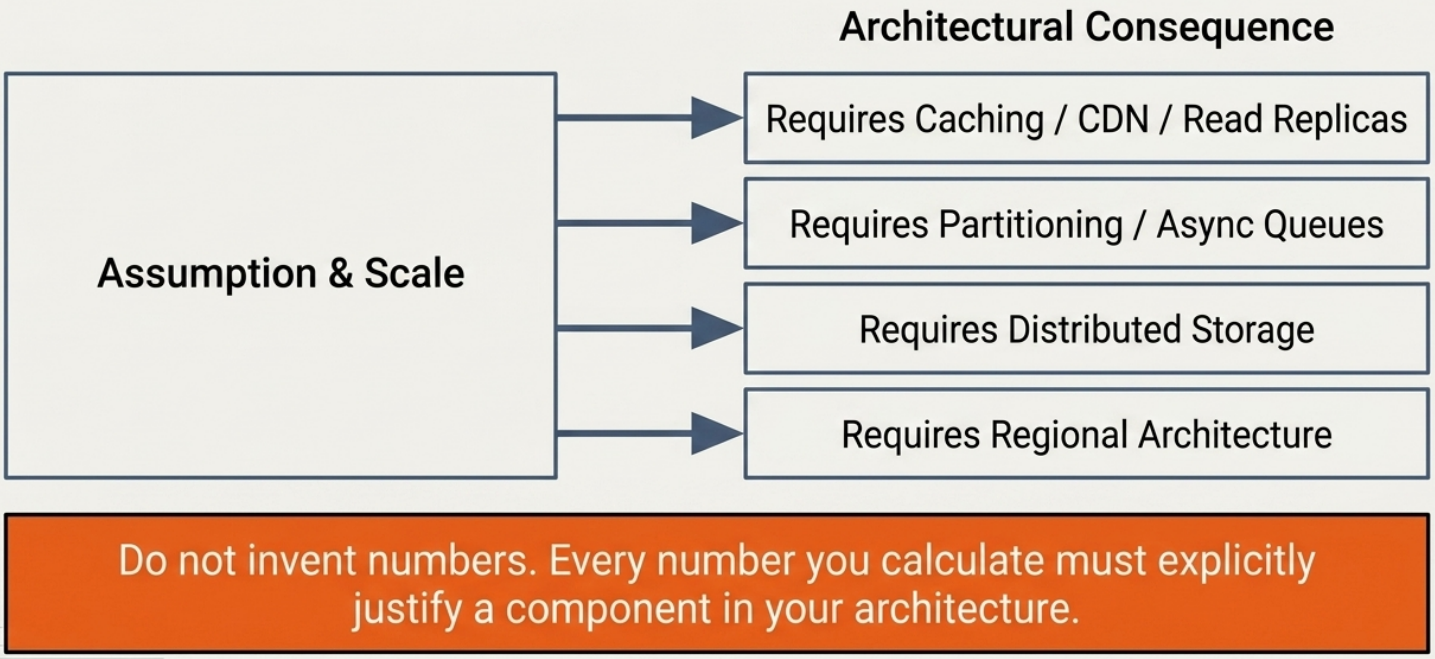
- Show countdown page
- Reserve exact stock
- Charge winning buyers exactly once

The Constraints (Non-Functional)

- Survive vertical traffic spike
- Zero overselling (Strict Consistency)
- Strict fairness (P99 latency < 300ms for queue)

A Flash Sale is a write-heavy, contention-heavy problem hiding inside a standard e-commerce page. The constraints dictate the design.

Phase 2 Playbook: Derive Numbers from Assumptions



Do not invent numbers. Every number you calculate must explicitly justify a component in your architecture.

Execution: The Lethal Math of a 10,000 Unit Drop

Given Variables

Concurrent users: 10,000,000

Available stock: 10,000

Read/Write ratio: 1,000 : 1

Sale window: ~60 seconds

Peak Deductions

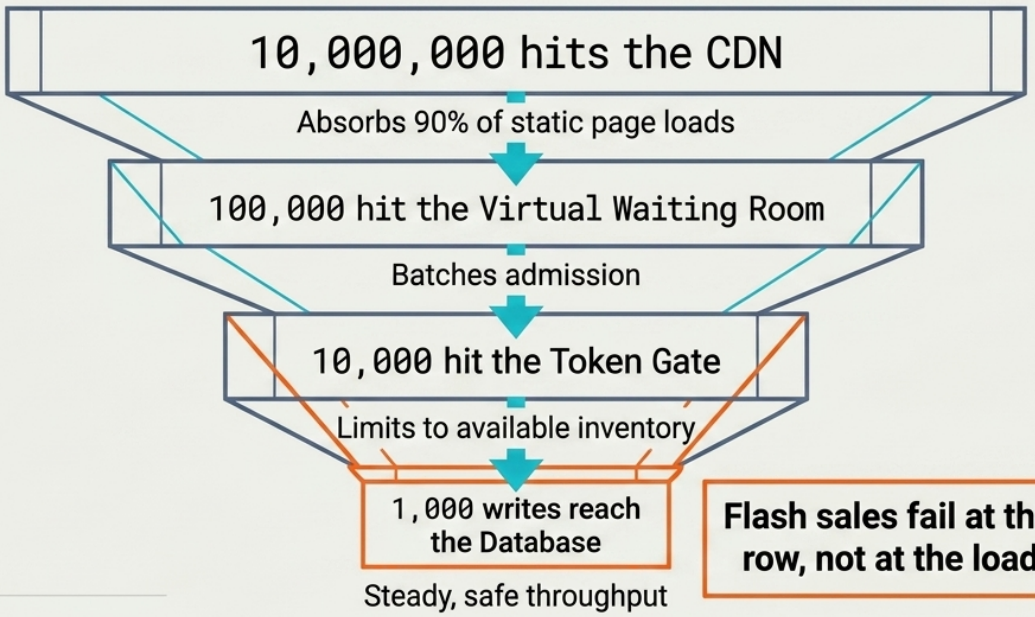
Peak page views: 500,000 reads/sec (Dominates the read path)

Peak order writes: 1,000 writes/sec (Lethal contention on a single DB row)

Storage: ~5 MB total (Storage size is irrelevant here)

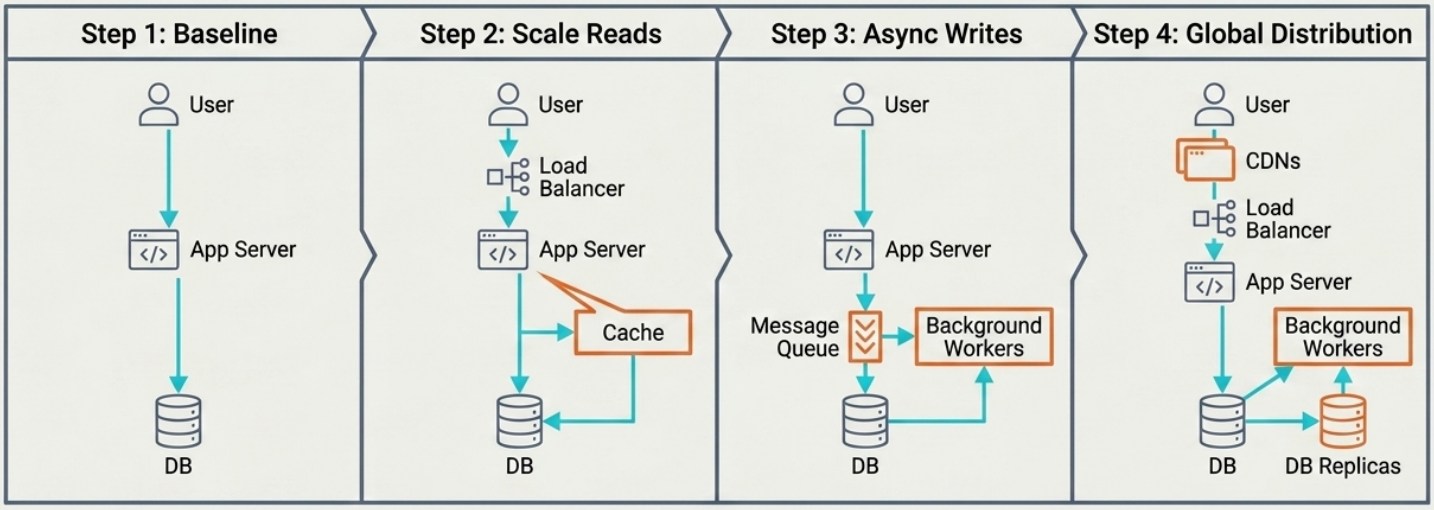
The Order of Magnitude Funnel

Goal: Reduce the vertical spike to a trickle before it hits the database.



Flash sales fail at the inventory row, not at the load balancer.

Phase 3 Playbook: Build Incrementally



Never add a component without a reason. Start with the simplest viable system. Introduce complexity only to solve the bottlenecks identified by your numbers.

Always Define APIs and Schema Before Drawing Boxes

API Design

- Method & Endpoint
- Request payload
- Response payload
- Idempotency considerations

Data Model

- Primary key
- Important fields
- Table relationships
- Partition keys

Keep APIs focused on core requirements. This bounds the problem and proves you understand the data flow.

Execution: The Flash Sale Interface

```
GET /sales/:id
Returns sale metadata.
Cache-Control: max-age=10.
```

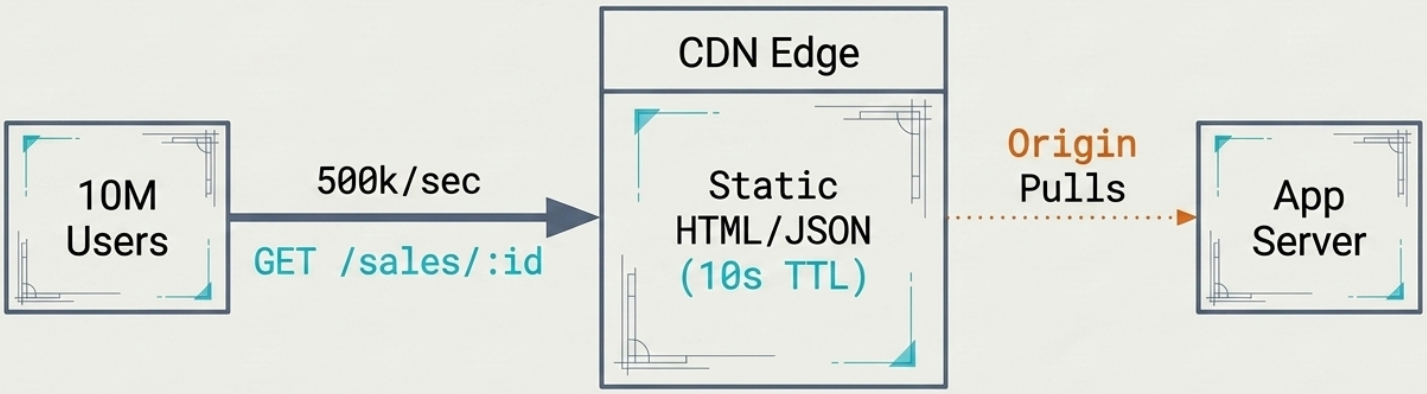
```
POST /sales/:id/enter
Waiting room entry.
Returns 202 Accepted with a queue_token.
```

```
POST /sales/:id/orders
Requires: Idempotency-Key and X-Queue-Token
Returns: 202 Accepted (Order is queued, NOT yet
        in the database).
```

Notice the **202 Accepted**. Synchronous work inside the user request kills flash sales.

Step 1: The Read Path

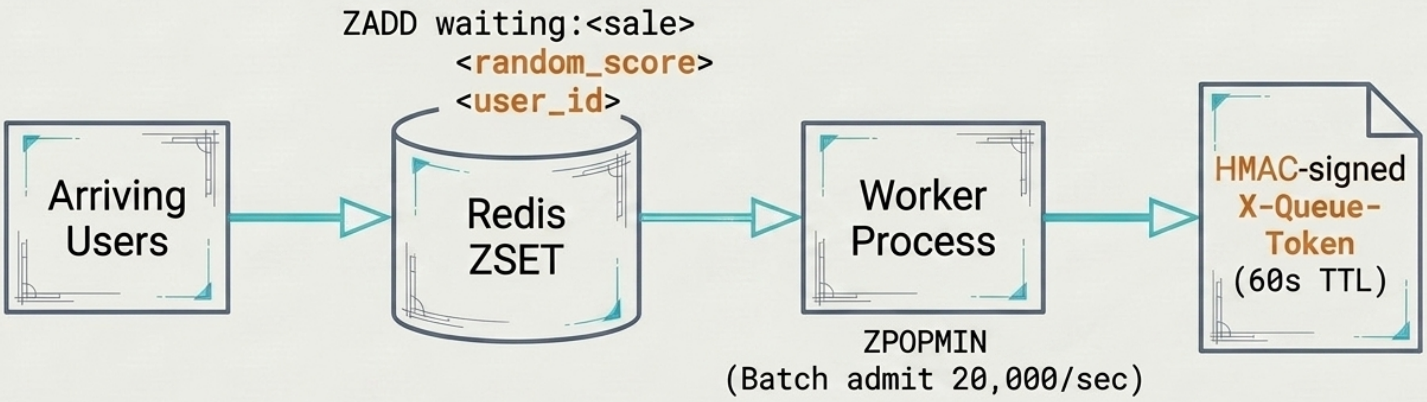
The cheapest request is the one that never reaches your servers.



The 500k/sec page views will melt the system if they hit the origin. Offload static countdown pages and metadata to the edge. The stock counter on the page is just an eventual consistency hint.

Step 2: The Virtual Waiting Room

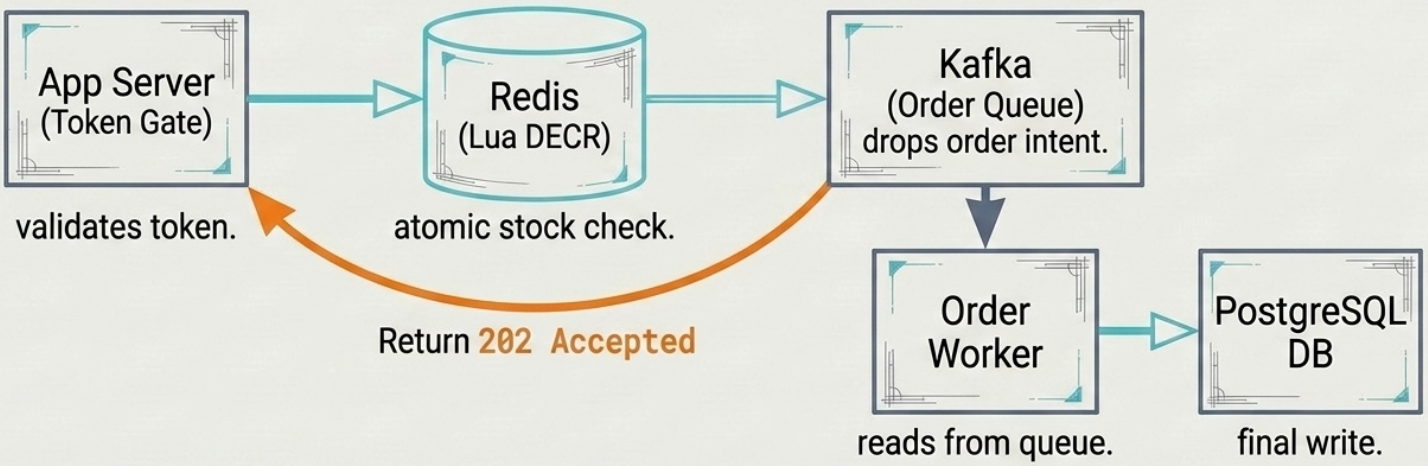
Clamping vertical traffic to a sustainable rate.



A random score gives every user a fair chance, defeating bots that optimize for timestamp-based FIFO.
The signed token ensures users cannot bypass the line.

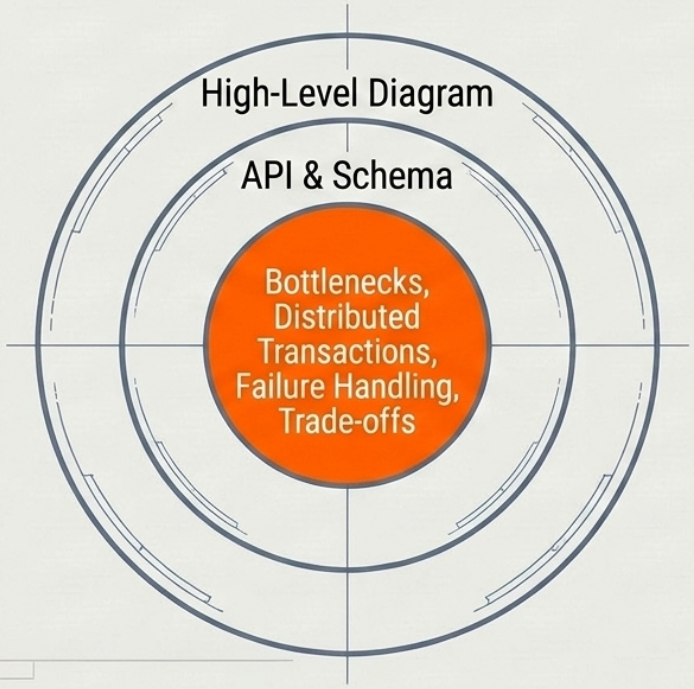
Step 3: The Async Order Pipeline

Decoupling checkout from DB writes.



Touch Redis, drop in Kafka, return to the user in single-digit milliseconds.
The worker scales independently based on queue depth.

Phase 4 Playbook: Go Deep Where It Matters

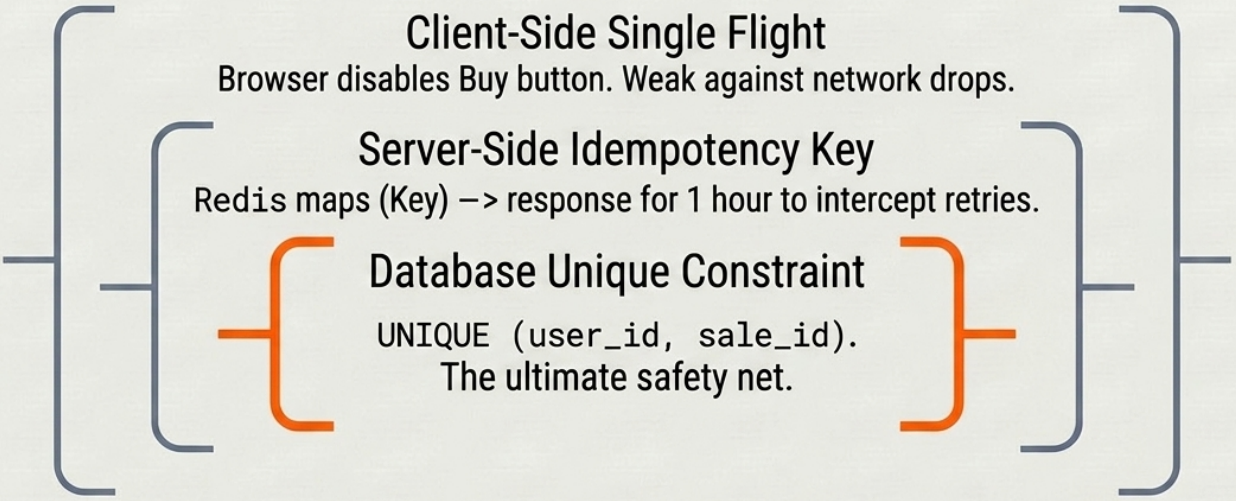


This is where you prove seniority. Do not wait for the interviewer to find the flaws in your system. Proactively identify the hardest technical challenges, compare trade-offs, and defend your choice.

The Hardest Problem: Four strategies to prevent overselling

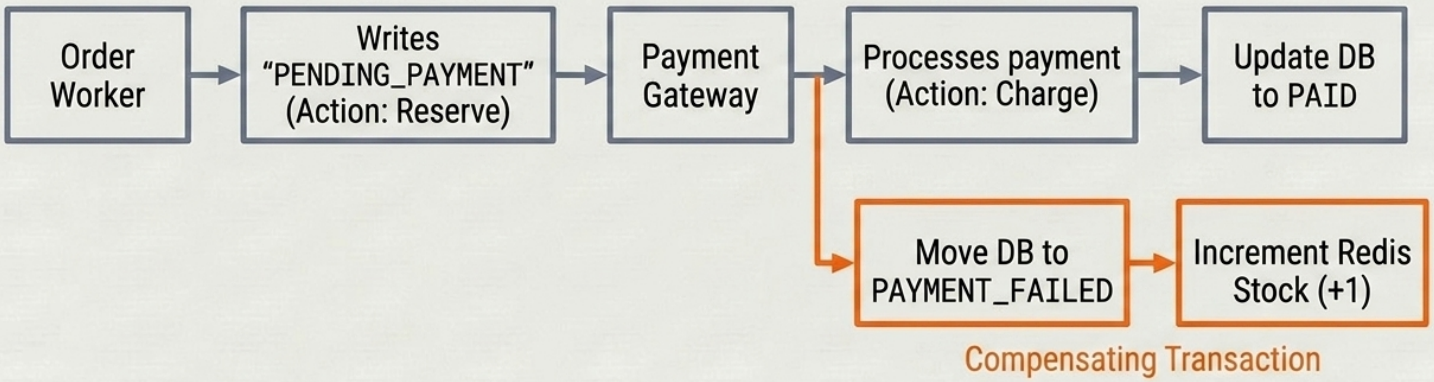
Strategy	Throughput & Correctness	Failure Mode / Verdict
Pessimistic Lock (`FOR UPDATE`)	`100s/sec | Strict`	DB tip-over. Do not use for Flash Sales.
Optimistic Lock (`WHERE stock > 0`)	`5k–10k/sec | Strict`	Retry storms / Thundering herd.
Redis Lua DECR	`100k/sec | Strict on single node`	Memory loss. Standard industry choice.
Token Gate (Pre-minted)	`100k/sec | Mathematically strict`	Token list corruption. Use for extreme bot pressure.

Defense in Depth: Preventing Duplicate Orders



Duplicate orders happen because networks are unreliable.
If the DB unique constraint fires, something upstream broke.

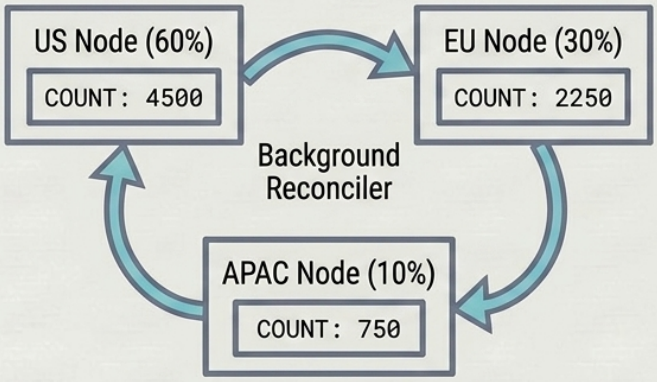
Distributed Transactions: Reserve, Charge, Settle Saga



Never hold a sale hostage for a payment gateway. Give a 10-minute soft hold. The compensating transaction is critical to avoid locked, unsold inventory.

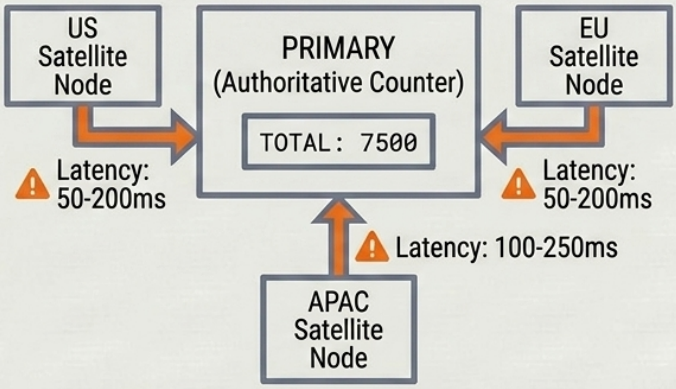
Global Scaling: Multi-region Architecture Trade-offs

Sharded with Rebalancing



Trade-off: Eventual consistency, high throughput. Standard for e-commerce.

Single Authoritative Counter



Trade-off: Strict global fairness, but incurs cross-region latency (50-200ms). Used for globally rare drops.

Anticipating Failure Modes

The senior move is predicting how it breaks.

Failure	Symptom	Architectural Fix
Redis primary memory loss	Counter resets, oversell occurs	**AOF** appendfsync everysec, → **Sentinel** sync replication
Thundering herd on cache miss	App servers hammer Redis simultaneously	Single-flight per app server, → short negative cache TTL
Bot wave drains stock in 50ms	99% bot orders, humans excluded	Token bucket limits, → edge → WAF, Account-age gating

The Complete Blueprint

Integrated architectural master diagram for high-throughput systems